FUTURISTIC AI CLASSROOM & ORGANIZATION MANAGEMENT SYSTEM

Production architecture, feature roadmap, engineering decisions, security, scaling, disaster recovery and execution checklists

Technology: HTML, CSS, JavaScript, Django, PostgreSQL, Redis, Celery, WebRTC SFU, object storage and a self-hosted AI service.

Planning edition: 10 November 2026 | **Intended deployment:** multi-branch, initial 100-1,000 active users with growth beyond that.

1. Executive recommendation

Build a **modular Django monolith** for core academy business logic and separate resource-intensive services for video transport, recording, AI inference, and messaging. Do not start with a microservice for every module. Establish tenant isolation, authorization, automated tests, and backups before advanced AI.

Feasibility: Custom real-time classes, meetings, recording, targeted messaging and document-based question/answer training are feasible. A proprietary AI model trained entirely from scratch is a distinct research and data/compute program; it should not block the platform MVP.

2. Users, tenancy and permissions

Scope	Roles / capabilities
Platform	Super admin: organization provisioning, policies, billing, platform health; no implicit access to private student data without audited support workflow.
Organization	Organization owner, organization admin: branches, plans, cross-branch reporting and organization-level settings.
Branch	Branch manager, office staff: local schedules, admissions, student roster, communications and operations.
Learning	Trainer/assistant, student, parent/guardian: assigned batches, attendance, content, exams and progress.
Operations	Support, compliance, auditor: limited, audited access only.

Tenant design: Organization -> Branch -> Course -> Batch -> Enrollment. Every tenant-owned record must carry an organization association; branch association where applicable. Querysets, API endpoints, background jobs, websocket tokens and file access must enforce scope server-side. Include cross-tenant access tests.

3. Feature catalogue

Subsystem	Capabilities
Organization control	Multi-organization management; branches; departments; academic years; roles; invitations; staff transfers; centralized metrics; audit history.
Academic operations	Courses, curriculum, batch timetable, enrollment, waiting lists, classroom resources, attendance, leave, assignments, exams, grades, certificates.
Finance & CRM (later)	Admissions funnel; leads; fee schedules; invoices; payment status; discounts; refunds; settlements; consent and receipts.
Live classroom	Scheduled/ad-hoc rooms; trainer/moderator roles; screen sharing; chat; raise hand; waiting room; mute controls; breakout rooms (later).
Recordings & library	Opt-in recording with clear notice; processing; thumbnails; transcript/captions (later); storage retention; enrolled-student playback; expiry.
Communication	In-app center; segmented recipient selection; announcements; email; WhatsApp approved templates; delivery status; retry; opt-out.
AI learning	Upload PDFs/DOCX/notes; text extraction/OCR fallback; source-linked question generation; interview practice; rubric-based grading; targeted corrections.
Intelligence (later)	Knowledge gaps; personalized study queue; early-warning signals; branch analytics; instructor coaching indicators with human review.
Support & governance	Ticketing; incident management; audit logs; access review; retention/deletion; backups; admin operational dashboard.

4. Reference system architecture

Client: Server-rendered Django templates + HTML/CSS/vanilla JavaScript initially. Use small JavaScript modules or HTMX where needed; add a SPA only if the product complexity justifies it.

Ingress: DNS/CDN/WAF -> HTTPS reverse proxy/load balancer -> multiple Django ASGI application instances.

Business services: Django accounts, organizations, branches, academics, learning, assessments, notifications, media catalog, reporting, billing (optional). Django REST Framework for APIs; Channels/WebSockets only where the user experience requires them.

Supporting services: PostgreSQL (source of truth); Redis (cache/queues/channel layer); Celery workers/scheduler (async tasks); S3-compatible object storage (private recordings/documents); search index or PostgreSQL full-text for retrieval; dedicated AI workers.

Media plane: Browser camera/microphone -> WebRTC -> SFU (e.g., self-hosted LiveKit). Django authenticates users and issues short-lived scoped room tokens. Separate egress/recording workers write media to private object storage. Provide TURN for restrictive networks.

Operational plane: Central logging, metrics, tracing, alerting, container images, CI/CD, staging and recovery environment. Secrets managed outside the repository.

Suggested Django modules

accounts, organizations, branches, academics, admissions, enrollments, attendance, assignments, exams, live_classes, recordings, materials, ai_assessments, notifications, analytics, audit, support, billing (deferred).

Representative entities

Domain	Models / fields to prioritize
Tenancy	Organization, Branch, Membership, RoleAssignment, PermissionGrant, AuditEvent
Learning	Course, Batch, TrainerAssignment, Enrollment, Session, Attendance, Assignment, Submission, Exam, Attempt
Video	LiveRoom, RoomParticipant, RecordingJob, RecordingAsset, RecordingAccessLog
AI	Document, DocumentPage, ExtractionJob, Question, EvaluationRubric, PracticeAttempt, AnswerFeedback, ModelVersion
Messaging	AudienceSegment, MessageTemplate, NotificationCampaign, Delivery, ConsentRecord
Operations	BackupRun, RestoreDrill, SecurityEvent, Incident, ServiceHealth

Important authorization examples

- A student can access only documents, rooms and recordings permitted by active enrollment and access policy.
- A branch manager cannot read or modify another branch unless explicitly granted scope.
- A recording link must not disclose a public object-storage key; use authenticated checks and short-lived signed URLs.
- Role changes and student unenrollment invalidate future join/playback authorization promptly.

5. Live meetings and live classes

Use a WebRTC Selective Forwarding Unit (SFU) instead of transmitting raw video through Django. Django manages scheduling, room authorization and attendance events; the SFU manages media streams. LiveKit supports self-hosting, separate egress recording, and TURN/network configuration.

■ Trainer schedules a session and selects branch, batch, participant capacity and recording policy.

■ Backend validates trainer ownership, session time, enrollment and simultaneous room policy.

■ Backend creates room metadata and generates short-lived role-scoped WebRTC token.

■ Users join on camera/microphone permission; controls enforce moderator/student capabilities.

■ Webhooks capture room start/end, participants and recording progress; make handlers idempotent.

■ Egress worker records permitted sessions; completion job verifies file integrity and writes storage metadata.

■ Playback endpoint re-checks enrollment, retention policy and organization scope on every request.

Capacity warning: 1,000 active website users are not equivalent to 1,000 simultaneous video participants. Test numbers of rooms, participants per room, send/receive bitrates, TURN relay ratios, concurrent recording jobs and geographic network quality independently.

6. AI interview learning workflow

Document pipeline: Upload -> virus/type/size check -> secure object storage -> extract text and page numbers -> normalize/chunk -> retrieval index -> question generation -> validation -> instructor review/publish -> student practice -> answer evaluation -> explanations and source citations.

Question types: factual, conceptual, scenario-based, coding, troubleshooting, behavioral interview and progressively difficult follow-ups. Question metadata: source pages, topic, skill, difficulty, rubric and model version.

Evaluation: Check relevance against source text, key concepts, omissions, factual correctness and clarity using a versioned rubric. Return score, supported strengths, missing ideas, corrected model answer, explanation and document page references. Flag uncertain grading for teacher review.

Reality check on an original AI model

Training a reliable general-purpose language model from scratch requires large licensed datasets, compute, expertise and evaluation. A realistic first release uses deterministic extraction/retrieval and rule/rubric-based or locally hosted language generation. If original weights are mandatory, scope them to a narrow component first (for example difficulty classifier, topic tagger or answer-quality scorer), collect labeled data, train a small model and benchmark it. This does not yet replace a high-quality generative model.

- Specify exactly which components must use entirely original weights versus locally hosted open-weight models.
- Establish consent/licensing and permitted retention for uploaded study materials.
- Build evaluation dataset: 100+ reviewed examples across topics, difficulty and answer quality as an initial pilot target.
- Measure unsupported assertions, answer correctness, page attribution, grading consistency and response latency.
- Prevent cross-organization document retrieval, prompt injection from uploaded text and untrusted file execution.
- Require human override, appeal flow and transparent AI-assisted grading disclosure.

7. Messaging and notifications

Target audiences using organization, branch, batch, course, enrollment status and role. Queue messages via Celery; never send large broadcasts synchronously in an HTTP request.

- Email sender/domain authentication, templates, bounce processing and unsubscribe rules where appropriate.
- WhatsApp Business Platform integration using approved templates for business-initiated notifications, recorded opt-in and opt-out handling.
- In-app notification inbox with read/unread state, language preferences and deep links.
- Campaign preview: exact recipient count, suppression reasons, deduplication and approval for bulk sends.
- Rate limits, provider retries/backoff, delivery webhooks, budget controls and audit logs.

WhatsApp is not a free unrestricted bulk-messaging pipe. Business-initiated messages require approved templates; platform rules, user choice and pricing apply.

8. Security, privacy and governance

- Least-privilege RBAC, object-level permissions and organization/branch isolation throughout web/API/media/AI workflows.
- MFA for privileged users; secure password resets; session/device revocation and rate-limited authentication.
- HTTPS/TLS, secure cookies, CSRF, security headers, encrypted backups and secrets rotation.
- Private buckets, malware scanning, file size/type restrictions, signed playback URLs and access logging.
- Consent for class recording; visible recording indicator; policies for minors, parents, retention and deletion.
- Document local legal requirements before launch, including applicable Indian data protection obligations.

■ Audit security-relevant actions, test dependency vulnerabilities and run Django deployment checks.

9. Scalability plan: 100 -> 1,000 -> beyond

Load tier	Practical implementation
Pilot / ~100 active users	1-2 Django instances, managed PostgreSQL or well-operated database, Redis, single worker pool, object storage, modest SFU node. Load-test before promising concurrency.
Growth / ~1,000 active website users	Stateless Django replicas behind load balancer; DB connection pooling; query/index review; Redis caching; separate worker queues; autoscale based on measured CPU/latency/queue depth.
Video-intensive growth	Multiple SFU hosts/regions as needed; bandwidth and CPU budgets; TURN monitoring; recording workers sized per concurrent recording; admission limits and graceful degradation.
Enterprise / multiple organizations	Strong tenant isolation; dedicated analytics/report queues; read replicas if justified; capacity planning; regional compliance; SLOs and incident on-call.

Performance targets (proposed, not guarantees): 95th percentile non-video API response under 500 ms for common endpoints under agreed test load; 99.9% monthly availability initial target; scheduled jobs complete within defined windows. Establish separate media-quality and AI-latency targets after pilot measurements.

Load testing scenarios

- 100, 300 and 1,000 authenticated web users: login, dashboards, enrollments, attendance and material download.
- Peak write contention: concurrent attendance edits, enrollment capacity and exam submissions.
- Video scenarios: 10x20, 20x30 and 10x100 participants as separate illustrative test matrices, not assumed supported capacity.
- Mixed loads: AI inference + messaging burst + video + recording + routine web traffic.
- Measure p50/p95/p99 latency, error rate, DB connections, CPU, memory, network egress, storage and queue delay.

10. Disaster recovery and continuity

Set business recovery objectives before launch. Proposed initial targets: **RPO <= 15 minutes** for core transactional database using tested WAL archiving/PITR, and **RTO <= 4 hours** for a major regional outage. These are design goals, not actual capabilities until recovery drills prove them.

Asset	Protection and recovery plan
PostgreSQL	Automated full/base backups plus WAL archives for point-in-time recovery; encrypted copies in a separate region/account; retention and restore verification.
Documents/recordings	Versioned or protected private object storage; cross-region/cross-account replication according to criticality; lifecycle and deletion controls.
Infrastructure	Infrastructure-as-code, immutable images, pinned dependencies, secure secrets repository and documented re-provisioning procedure.
Applications	Health checks, rolling/blue-green deployments where appropriate, controlled DB migrations, and tested rollback plans.
Communications	Incident runbook, operational contacts, user status updates and alternate notification route when one provider is unavailable.
DR exercises	Monthly restore verification initially; periodic full service failover drills; log measured RTO/RPO and action items.

Failure modes: host failure, database corruption, accidental deletion, ransomware, object-storage outage, message provider failure, SFU congestion, unavailable GPU and regional disaster. Decide in advance which features degrade: AI and video may pause while admissions, attendance history and notices remain available.

11. Phased delivery roadmap

Stage	Output / exit criteria
0 - Discovery and boundaries	Requirements, tenant rules, capacity scenarios, data classification, security/threat model, cost and acceptance criteria.
1 - Foundation	Multi-org and branches, roles, courses/batches, enrollments, attendance, audit, tests, CI/CD and daily backups. Release a usable non-video MVP.
2 - Live learning	Scheduling, WebRTC rooms, trainer controls, recording egress, secure access and media performance tests.
3 - Notifications	In-app, email and compliant WhatsApp workflow, segmentation, consent, webhooks and delivery reporting.
4 - AI learning	Upload/extract/index, question generation, interview attempts, scoring rubric, feedback, citations and teacher review.
5 - Advanced intelligence	Knowledge maps, personalized exercises, predictive analytics subject to evidence, multilingual features and advanced reporting.
6 - Hardening and scale	Concurrent-load validation, DR drills, access reviews, privacy readiness, observability, cost tuning and launch gates.

12. Master implementation checklist

A. Product and design

- ■ Document user journeys for every role and the branch hierarchy.
- ■ Define permissions matrix including cross-branch admin and guardian access.
- ■ Confirm release scope; explicitly defer nonessential futuristic ideas.
- ■ Design accessible responsive layouts: admin, trainer, student, parent.
- ■ Define retention, consent, financial and compliance policies.

B. Engineering foundation

- ■ Create Git repository, branch strategy, issue board and ADR folder.
- ■ Configure environment-specific Django settings, secrets and migrations.
- ■ Set PostgreSQL, Redis, task queue, object storage and structured logs.
- ■ Implement tenant-safe ORM managers/services and permission tests.
- ■ Configure CI checks for lint, unit/integration tests, security scans.
- ■ Set up staging, automated deployment and observability.

C. Academics and accounts

- ■ Organization / branch CRUD; invitations; role management.
- ■ Course, batch, trainer, student and enrollment workflows.
- ■ Timetable conflicts, attendance rules and instructor approvals.
- ■ Assignments, exams, results and learner progress pages.
- ■ Audit trails and safe export/reporting endpoints.

D. Video and content

- ■ Room scheduling, JWT/token scopes, moderator capabilities.
- ■ TURN and production SSL connectivity verified from different networks.
- ■ Recording notice, Egress worker and object storage permissions.
- ■ Playback access control, video retention and expiry jobs.
- ■ Video and recording failure/retry policies.

E. AI learning

- ■ Secure document uploads, extraction and page map.
- ■ Retrieval grounding, question/rubric schema and reviewed evaluations.
- ■ Student attempt, answer comparison, explanations and page citations.
- ■ Model/version tracking, quality benchmark and teacher override.
- ■ AI cost, throughput, data isolation and hallucination monitoring.

F. Notifications

- ■ Audience segmentation, opt-in records and unsubscribe/opt-out.
- ■ Email and approved WhatsApp template integration.
- ■ Background queues, rate limits, retries, failure reporting.
- ■ Bulk campaign approval, monitoring and audit.

G. Release readiness

- ■ python manage.py check --deploy passes with verified settings.
- ■ Tenant isolation and authorization penetration tests pass.
- ■ Load tests reproduce stated user and video capacity.
- ■ Backup restore and regional recovery drill meet RTO/RPO.
- ■ Metrics, on-call alerts, incident response and rollback are verified.
- ■ Pilot feedback, accessibility and mobile browser tests completed.

13. Initial sprint: tangible deliverables

Sprint 1: Repository skeleton; environment configuration; Organization/Branch/Membership models; scoped query policies; migrations; sample seed data; UI navigation by role; tests for tenant isolation and branch permissions; Docker Compose development environment; one verified backup/restore run.

Definition of done: Admin can create an organization and branch; invite a branch manager; manager can add batches and enroll learners only within permitted scope; student sees only permitted data; unauthorized cross-tenant queries fail; CI tests pass; a backup can be restored in a clean environment.

Build decision: For an existing EduCore codebase with these functions, inspect and reuse working modules, migrations and tests rather than recreate them. Start with an architecture audit and gaps list.

14. Risks, tradeoffs and key decisions

Decision	Recommendation and rationale
Custom video protocol vs own meeting UX	Build your own meeting UI and Django authorization; do not implement the WebRTC SFU from scratch unless media infrastructure is the actual product.
Modular monolith vs microservices	Modular monolith for business domain; isolate high-compute media and AI services. Split further only when measured constraints justify it.
From-scratch AI vs reliable initial product	Define original-weight requirement per component. Data ingestion and validated evaluation deliver product value before expensive large-model training.
Unlimited use vs admission control	Specify per-organization room, recording, storage and AI quotas; prevent one tenant saturating shared compute.
Recording by default vs privacy	Make recording policy explicit with consent/notice, approved access and retention periods. Never assume all sessions should be recorded.
Single-region simplicity vs DR	Start with a comprehensible deploy plus off-site backups; progress to warm standby and cross-region failover as the required RTO tightens.

15. Success metrics and acceptance gates

- Functionality: admission -> enrollment -> class -> attendance -> assignment -> results -> student progress works end-to-end.
- Security: 0 confirmed critical cross-tenant permission flaws; all known high-severity launch blockers resolved.
- Reliability: measured backup freshness and restore time match approved business recovery objectives.
- Live classroom: documented join success rate, acceptable quality threshold and peak concurrent test results.
- AI evaluation: reviewed test set measures factuality, grading agreement, citation validity and harmful/confidently-wrong feedback.
- Operations: incident alerts, logging, support escalation, release rollback and data export/delete processes are executable.

16. Source references and implementation reading

LiveKit self-hosting and Egress: https://docs.livekit.io/transport/self-hosting/ and https://docs.livekit.io/transport/self-hosting/egress/

LiveKit production deployment: https://docs.livekit.io/transport/self-hosting/deployment/

Django production checklist: https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

WhatsApp Business policy: https://whatsappbusiness.com/policy/

PostgreSQL continuous archiving and PITR: https://www.postgresql.org/docs/current/continuous-archiving.html

Note: Vendor capabilities, regulations, messaging pricing and hosting limits may change. Verify product versions and legal obligations before procurement and launch. The capacity numbers and delivery stages here are planning goals, not an assurance of production performance.